KUBERNETES CHEAT SHEET • *Senior DevOps Interview — Quick Reference* • Commands · Architecture · Debug · Production

ARCHITECTURE — CONTROL PLANE

```
┌──────────────────────────────────────┐
│ kube-apiserver  ←  ALL requests      │
│ etcd            ←  cluster state     │
│ kube-scheduler  ←  pod→node assign   │
│ controller-mgr  ←  reconcile loop    │
│ cloud-ctrl-mgr  ←  LB/EBS/etc        │
└──────────────────────────────────────┘
NODE: kubelet + kube-proxy + containerd
```

POD LIFECYCLE PHASES

Pending	Scheduled; waiting for image pull or node
Running	At least 1 container actively running
Succeeded	All containers exited 0 (Job done)
Failed	All containers exited; ≥1 non-zero exit
Unknown	kubelet lost contact — node unreachable

QoS CLASSES (eviction order: 3→1)

BestEffort	No requests/limits — evicted FIRST
Burstable	requests < limits — evicted 2nd
Guaranteed	requests == limits — evicted LAST

WORKLOAD CONTROLLERS

Deployment	Stateless apps — rolling update + rollback
StatefulSet	Databases — stable name, PVC per pod, ordered
DaemonSet	One pod per node: log agent, monitoring
Job	Run-to-completion batch task
CronJob	Scheduled jobs — cron syntax
ReplicaSet	Maintains N pod copies (owned by Deployment)

PROBE TYPES

startupProbe	Gives slow apps time to start; blocks liveness

KUBECTL — ESSENTIAL COMMANDS

— INSPECT & DEBUG

```
# All pods across namespaces with node info
kubectl get pods -A -o wide

# Full pod detail — always start here
kubectl describe pod <name> -n <ns>

# Logs (add -f to follow live)
kubectl logs <pod> -n <ns> -c <container>
kubectl logs <pod> --previous        # crashed
container

# Shell into running container
kubectl exec -it <pod> -n <ns> -- bash

# Ephemeral debug pod (network tools)
kubectl run dbg --rm -it \
  --image=nicolaka/netshoot -- bash

# Events sorted by time — gold for debugging
kubectl get events -A \
  --sort-by='.lastTimestamp' | tail -30
```

— DEPLOYMENTS

```
kubectl apply -f manifest.yaml
kubectl rollout status deploy/<name>
kubectl rollout history deploy/<name>
kubectl rollout undo deploy/<name>
kubectl rollout undo deploy/<name> --to-revision=3
kubectl scale deploy/<name> --replicas=5
kubectl set image deploy/<n> app=repo/img:tag
```

— RESOURCE & CONFIG

```
kubectl get nodes -o wide
kubectl top nodes
kubectl top pods -n <ns> --sort-by=cpu
kubectl get resourcequota -n <ns>
kubectl get limitrange   -n <ns>
kubectl get hpa          -n <ns>
kubectl get pvc,pv       -n <ns>
```

— SECRETS & CONFIGMAPS

```
# Decode a secret value
kubectl get secret <name> -o jsonpath=\
  '{.data.password}' | base64 -d

# Quick test ConfigMap
kubectl create cm test --from-literal=k=v

# Check RBAC permissions
kubectl auth can-i get pods --as <user> -n <ns>
```

DEBUGGING PLAYBOOK

CrashLoopBackOff

```
1. kubectl describe pod <n>  →  check Events
   Exit Code 137  =  OOMKilled (memory limit hit)
   Exit Code 1    =  App error (check logs)
   Exit Code 139  =  Segfault

2. kubectl logs <pod> --previous   ← most missed!

3. Root causes: bad env var, unreachable DB,
   misconfigured readiness, startupProbe too tight

4. Fix startupProbe: set failureThreshold: 30
   (30 × periodSeconds = grace window)
```

Pod PENDING

```
kubectl describe pod <n> | grep -A10 'Events'

"Insufficient cpu/memory"
  → add nodes, reduce requests, or delete idle pods

"No nodes match taints"
  → add tolerations or remove taint

"PVC not bound / unbound PV"
  → check StorageClass, AZ match, capacity

"Unschedulable: 0/3 nodes"
  → check nodeAffinity, nodeSelector labels
```

Service Not Reachable

```
# Step 1: Check endpoints (most common cause)
kubectl get endpoints <svc>
→ Empty = label selector mismatch!
  kubectl get pods -l 'app=<name>'

# Step 2: DNS resolution
kubectl exec <pod> -- \
  nslookup <svc>.<ns>.svc.cluster.local

# Step 3: Direct connectivity
kubectl exec <pod> -- \
  curl -v http://<svc>.<ns>:8080/health

# Step 4: NetworkPolicy blocking?
kubectl get networkpolicy -n <ns>
```

Node NotReady

```
kubectl describe node <n> | grep Conditions -A10
kubectl get events --field-selector \
  'involvedObject.name=<node>'
```

readinessProbe	Remove from Service endpoints if not ready
livenessProbe	Restart container if check fails

Rule: Never check DB in livenessProbe — cascading restarts!
Rule: readiness ≠ liveness endpoint — separate them always

SERVICE TYPES

ClusterIP	Internal cluster access only (default)
NodePort	NodeIP:port 30000–32767 — external (limited)
LoadBalancer	Cloud LB provisioned — production external
ExternalName	CNAME alias to external DNS
Headless	ClusterIP:None — direct pod IPs via DNS

— NODE MAINTENANCE

```
# Stop new pods landing on node
kubectl cordon   <node>

# Drain node safely before upgrade/maintenance
kubectl drain <node> \
  --ignore-daemonsets \
  --delete-emptydir-data

# Return node to service
kubectl uncordon <node>
```

— CONTEXTS & NAMESPACES

```
kubectl config get-contexts
kubectl config use-context <ctx>
kubectl config set-context --current \
  --namespace=<ns>

# Aliases that save hours in CKA exam:
alias k=kubectl
export do='--dry-run=client -o yaml'
source <(kubectl completion bash)
```

```
# Common reasons:
→ DiskPressure:    df -h on node, clean images
→ MemoryPressure:  OOM on node kernel
→ kubelet stopped: systemctl status kubelet
→ Network plugin:  kubectl logs -n kube-system \
     -l k8s-app=<cni-name>
```

PRODUCTION RULES

```
1.  minReplicas: 3   (never 1 in production)
2.  PodDisruptionBudget: minAvailable: 2
3.  preStop: {exec: {command: [sh,-c,sleep 15]}}
    → fixes LB deregistration race (502s on rollout)
4.  terminationGracePeriodSeconds: 60
5.  readinessProbe ≠ livenessProbe endpoint
6.  resources.requests set on EVERY container
7.  topologySpreadConstraints: spread across AZs
8.  Guaranteed QoS (req==lim) for databases only
9.  Never run as root: runAsNonRoot: true
10. Image tag = git SHA (never :latest in prod)
```

ETCD CRITICAL FACTS

```
3 nodes → quorum=2 (tolerates 1 failure)
5 nodes → quorum=3 (tolerates 2 failures)
Lose quorum → API server READ-ONLY

etcdctl endpoint health --cluster
etcdctl snapshot save   /backup/snap.db
etcdctl snapshot restore /backup/snap.db \
  --data-dir=/var/lib/etcd-restore

NVMe SSD required — fsync latency kills perf
```

KUBERNETES CHEAT SHEET | *PAGE 2 — Networking · Storage · Security · Scaling · Helm*

NETWORKING — CNI · DNS · NetworkPolicy

CNI Plugins

Plugin	Best for
Calico	BGP/VXLAN, NetworkPolicy, enterprise
Cilium	eBPF, no iptables, L7 policy, 5k+ nodes
Flannel	Simple overlay, dev/test only
AWS VPC	EKS native — pods get real VPC IPs

CoreDNS & Service Discovery

```
FQDN: <svc>.<ns>.svc.cluster.local

Same namespace:  curl http://payments
Cross-namespace: curl http://pay.billing
Full FQDN:       pay.billing.svc.cluster.local

 ndots:5 causes 4 extra DNS lookups for
  short names before trying external.
  Fix: use trailing dot → db.prod.

Debug DNS:
  kubectl exec pod -- \
    nslookup kubernetes.default
  kubectl logs -n kube-system \
    -l k8s-app=kube-dns
```

NetworkPolicy — Default Deny

```
# 1. Deny all traffic in namespace:
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]

# 2. Allow frontend → backend :8080
spec:
  podSelector:
    matchLabels: {app: backend}
  ingress:
  - from:
    - podSelector:
        matchLabels: {app: frontend}
    ports: [{port: 8080}]

 Always allow DNS: egress port 53 UDP+TCP
```

STORAGE — PV · PVC · StorageClass · CSI

Access Modes

RWO	ReadWriteOnce — 1 node (EBS default)
ROX	ReadOnlyMany — many nodes read
RWX	ReadWriteMany — many nodes write (EFS/NFS)
RWOP	ReadWriteOncePod — 1 pod only (K8s 1.22+)

Reclaim Policy

Retain	PV kept — manual reclaim, no data loss
Delete	PV + storage deleted with PVC

PV Lifecycle

```
Available → Bound → Released → Failed
```

StorageClass — WaitForFirstConsumer

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
provisioner: ebs.csi.aws.com
volumeBindingMode:
  WaitForFirstConsumer  # ← AZ correct
parameters:
  type: gp3
  encrypted: "true"
```

PVC Debug Checklist

```
kubectl describe pvc <name>
→ No StorageClass match
→ EBS in wrong AZ for pod
→ Access mode not supported
→ Quota exhausted in namespace
```

SECURITY — RBAC · PSA · Secrets

RBAC Mental Model

```
WHO (ServiceAccount/User/Group)
 + CAN DO WHAT (verbs:
get,list,create,delete)
 + ON WHAT (resources:
pods,secrets,deploys)

Role = namespaced scope
ClusterRole = cluster-wide scope
Binding = attaches role to subject

# Test permissions:
kubectl auth can-i get secrets \
  --as system:serviceaccount:ns:sa -n ns
```

Pod Security Admission (K8s 1.25+)

```
# Label namespace to enforce:
pod-security.kubernetes.io/enforce:
restricted

restricted requires:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  capabilities: {drop: [ALL]}
  seccompProfile: {type: RuntimeDefault}
  readOnlyRootFilesystem: true
```

Secrets — Production Approach

```
Base64 ≠ encryption — plain text in etcd!
```

```
Production options:
  External Secrets Operator + AWS SM
  Sealed Secrets (encrypted in Git)
  Vault Agent sidecar injection

Always mount as FILE not env var
Encrypt etcd at rest (EncryptionConfig)
```

AUTOSCALING — HPA · VPA · KEDA · Karpenter

HPA v2 — CPU + Custom Metrics

```
apiVersion: autoscaling/v2
spec:
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
```

Formula: ceil(current × currentMetric ÷ target)

HPA + VPA on same metric = conflict — avoid!

KEDA — Scale to Zero

```
triggers:
- type: aws-sqs-queue
  metadata:
    queueURL: https://sqs...
    queueLength: "10"

minReplicaCount: 0   # true zero!
maxReplicaCount: 50
```

Cluster Autoscaler vs Karpenter

	CA → Karpenter
Scale time	2–3 min → ~30 sec
Node types	Fixed groups → any instance
Bin-packing	Manual → auto consolidation
Cost	Higher → Spot optimization

HELM & GITOPS

Helm Commands

```
helm install <rel> ./chart \
  -n <ns> -f values-
prod.yaml \
  --set image.tag=$SHA

helm upgrade --install <rel>
./chart
helm rollback <rel> 2   -n
<ns>
helm history <rel>      -n
<ns>
helm template <rel> ./chart
# dry-run
helm list -A
helm uninstall <rel>    -n
<ns>
```

ArgoCD GitOps

```
syncPolicy:
  automated:
    prune: true      #
delete removed resources
    selfHeal: true   #
revert manual kubectl!

Sync wave ordering:
  argocd.argoproj.io/sync-
wave: '0' # DB migrate
  argocd.argoproj.io/sync-
wave: '1' # app deploy

Rollback = git revert + push
ArgoCD re-deploys old state
automatically
```

Naim Khan | DevOps Engineer | https://www.linkedin.com/in/naim-khan-282b98235/

KUBERNETES CHEAT SHEET | *PAGE 3 — HA Design · Observability · Incidents · Interview Patterns*

HA CLUSTER DESIGN — 5 LAYERS

Layer 1 — Control Plane

```
3 control-plane nodes across 3 AZs
etcd: 3 nodes (Raft, quorum=2)
API server behind ALB (health-checked)
scheduler + ctrl-mgr: leader election
```

Layer 2 — Worker Nodes

```
Nodes spread across 3 AZs
podAntiAffinity: hostname — no 2 replicas
  on same node
topologySpreadConstraints: zone spread
Karpenter: auto-replace failed nodes
```

Layer 3 — Application

```
minReplicas: 3  (one per AZ minimum)
PodDisruptionBudget: minAvailable: 2
readinessProbe: remove from LB if unhealthy
preStop sleep 15 → LB deregister race fix
terminationGracePeriodSeconds: 60
```

Layer 4 — Data

```
EBS: WaitForFirstConsumer (correct AZ)
EFS: multi-AZ shared storage (RWX)
etcd: dedicated NVMe SSD + defrag CronJob
etcd snapshots → S3 every 6h (Velero)
```

Layer 5 — Network

```
ALB: multi-AZ by default (AWS LB Ctrl)
CoreDNS: 3 replicas with anti-affinity
Default-deny NetworkPolicy per namespace
Cilium Hubble: network observability
```

ZERO-DOWNTIME ROLLOUT CHECKLIST

```
1.  strategy:
      type: RollingUpdate
      maxUnavailable: 0       # never reduce
      maxSurge: 1             # 1 extra pod

2.  readinessProbe defined    # LB gate

3.  minReadySeconds: 10       # pause after ready

4.  lifecycle:
      preStop:
        exec: {command: [sh,-c,sleep 15]}
    → WITHOUT this: ~0.1% 502s per rollout!

5.  terminationGracePeriodSeconds: 60

6.  PodDisruptionBudget: minAvailable: 2
```

OBSERVABILITY — 4 GOLDEN SIGNALS

Signal	Metric to watch
Latency	p99 request duration (histogram_quantile)
Traffic	Requests per second (rate)
Errors	5xx rate / exception rate
Saturation	CPU throttle %, memory %, disk fill %

Critical Prometheus Queries

```
# HTTP error rate (5xx)
rate(http_requests_total{code=~'5..'}[5m])
  / rate(http_requests_total[5m])

# CPU throttle % — THE metric for CPU limits
rate(container_cpu_cfs_throttled_seconds[5m])
  / rate(container_cpu_cfs_periods[5m]) * 100

# Memory (K8s uses working_set for OOM, not usage)
container_memory_working_set_bytes

# Pod restart alert
increase(
  kube_pod_container_status_restarts_total[1h]
) > 5
```

kube-prometheus-stack: Grafana dashboards 315, 6417, 1860

SCHEDULER: FILTER → SCORE PHASES

Phase 1 — Filter (hard constraints, eliminates nodes)

```
Insufficient CPU or memory requests
Taint without matching toleration
nodeSelector / nodeAffinity mismatch
PVC AZ mismatch (EBS is zone-scoped)
Hard pod anti-affinity rule violated
```

Phase 2 — Score (soft preferences, ranks remaining)

```
Preferred nodeAffinity / pod affinity
Node with most available resources
Least pods of same type (spread)
Pod chosen for highest-score node
```

COST OPTIMIZATION

```
1.  Right-size: VPA + Goldilocks     30-50% savings
2.  Spot/Preempt via Karpenter       60-70% savings
3.  KEDA scale-to-zero (dev nights)   60% savings
4.  gp3 vs gp2 EBS                   20% cheaper
5.  VPC Endpoints: avoid NAT GW       $ per GB
```

INCIDENT RESPONSE PLAYBOOK

Universal 5-Command Start

```
kubectl get pods -A | grep -v 'Running\|Completed'
kubectl get nodes
kubectl get events -A \
  --sort-by=lastTimestamp | tail -30
kubectl top nodes
kubectl describe pod <name-of-failing-pod>
```

OOMKill (Exit 137) Investigation

```
1. kubectl describe pod → "OOMKilled"
2. Check limit: kubectl get pod -o yaml \
     | grep -A3 memory
3. Growing memory (leak) OR instant kill?
4. JVM issue? Add:
     -XX:MaxRAMPercentage=75.0
   JVM ignores cgroups without this flag!
5. Use VPA recommendations to right-size
6. Monitor: container_memory_working_set_bytes
   NOT usage_bytes — K8s uses working_set for OOM
```

CPU Throttling vs High CPU

```
High CPU   = process consuming lots of cycles
CPU Throttle = cgroup PAUSING the process
  → latency spikes even with free node CPU
  → REMOVE CPU limits for latency-sensitive svcs

# Detect throttling:
container_cpu_cfs_throttled_seconds_total
```

StatefulSet Pod Stuck on NotReady Node

```
 StatefulSet pods do NOT auto-reschedule!
  (unlike Deployment pods)

# Force delete to reschedule:
kubectl delete pod <pod> \
  --force --grace-period=0
```

SENIOR INTERVIEW PATTERNS

Q: Explain K8s networking model

```
4 fundamental rules:
1. Pod↔pod: no NAT, direct IP to IP
2. Node↔pod: no NAT
3. Pod sees its own IP (no masquerade)
4. Services: abstracted via kube-proxy/eBPF
```

Q: What happens on kubectl apply?

```
kubectl → API Server (authn + authz + admission)
        → etcd (persists desired state)
        → Deployment ctrl → ReplicaSet
        → Scheduler (picks node)
```

```
7.  minReplicas: 3  (multi-AZ coverage)
```

DEPLOYMENT STRATEGIES

Strategy	Trade-off
Rolling Update	Gradual, no extra infra, medium rollback speed
Blue-Green	Instant switch, double infra cost during deploy
Canary	Real-traffic % test, slowest but safest rollout
Recreate	Downtime guaranteed — dev/test only

```
6.  S3 lifecycle: gp→IA→Glacier tiers
7.  Delete: unattached EBS, idle LBs,
            stale ECR images (lifecycle policy)

Tool: Kubecost / OpenCost — per-namespace billing
```

INIT CONTAINERS & SIDECARS

```
initContainers: run to COMPLETION first
  - wait-for-db: nc -z mysql 3306
  - run-migrations: python manage.py migrate

K8s 1.29+ Native Sidecars:
  restartPolicy: Always (outlives app)
  Use for: log agent, service mesh, secret refresh
```

```
        → kubelet (starts container)
        → containerd → container running
```

Q: iptables scaling problem?

```
iptables: O(n) rule scan per packet
10k services → 100k rules → measurable latency

Solutions:
  kube-proxy IPVS mode  O(1) hash tables
  Cilium eBPF           zero iptables at all
```

CKA / CKAD EXAM QUICK-WINS

```
export do='--dry-run=client -o yaml'
export now='--force --grace-period=0'

kubectl run pod1 --image=nginx $do > pod.yaml
kubectl create deploy d1 --image=nginx $do
kubectl delete pod stuck-pod $now

# Copy pod spec quickly:
kubectl get pod <n> -o yaml > base.yaml

# CKS extras:
trivy image nginx:latest --severity HIGH,CRITICAL
falco --list | grep syscall
```